BVRIT HYDERABAD

College of Engineering for Women

Department of CSE(AI&ML)



CASE STUDY

Hospital Management System

Database Management System

Under the guidance of

Mrs V. Asha(Assistant Professor)

TEAM : 2

25WH1A6666 – Kamatam Shraddha

25WH1A6673 - Bandarupalli Bhuvana Kruthi

25WH1A6680 - Sree Nidhi Chintapalli

25WH1A6687 – Battu Navya

25WH1A6694 - Mudireddy Vishvateja

25WH1A66A1 – Rupika gouri Dasamantharao

25WH1A66A8 - Kaparapu Jahnavi

25WH1A66B6 - Kandlagunta Maheswari

25WH1A66C3 - Bail Kammara Pranitha

26WH5A6609 - Guddeti Srivani

INDEX

S.NO.	TOPIC	PAGE NO.
1	Introduction	1
2	Requirement Analysis	5
3	ER Diagram	12
4	Database Implementation	15
5	SQL Queries and Outputs	27

- ROLLBACK should cancel uncommitted changes when an operation fails.
- Transaction handling should prevent partial updates and inconsistent records across related modules.
- Appropriate transaction isolation and locking mechanisms should be used where concurrent operations could create conflicts.

1.1.1 Auditability

- Clinical and financial records should identify the staff members responsible for relevant entries or transactions wherever applicable.
- Stored transaction information should support the review of billing, prescribing, dispensing, and administrative activities.
- Critical database modifications should be traceable through appropriate audit logs.
- Audit records should capture relevant information, such as the responsible user, operation performed, and timestamp.
- Access to audit logs should be restricted to authorized personnel, and records should be protected against unauthorized modification.

1.2 Integration Requirements

Integration requirements define how different components of the Hospital Management System exchange information and maintain consistency through the MySQL database.

1.2.1 Key and Referential Integrity Requirements

- Each table should have an appropriate primary key to uniquely identify its records.
- Foreign keys should connect related tables, such as linking appointments to patients and doctors, and prevent invalid references.
- Junction tables connecting multiple entities should use appropriate primary keys, which may be composite or separate unique identifiers depending on the database design.
- Related records should maintain referential integrity during insertion, updating, and deletion operations.
- Referential actions such as RESTRICT, CASCADE, or SET NULL should be selected according to the requirements of each relationship.

1.2.2 Patient and Appointment Integration

- Each appointment should be associated with an existing, valid patient and doctor record.
- Patient history should reflect past and upcoming appointments accurately.
- Changes to appointment dates, times, or statuses should be reflected in appointment retrieval and reporting operations.
- Appointment scheduling should validate doctor availability and prevent conflicting bookings through appropriate validation and transaction controls.

1.2.3 Doctor and Prescription Integration

- Prescriptions should be associated with valid doctor records and relevant patient consultations or visits.
- Prescription items should identify the medicines prescribed through valid medicine references.
- Prescription and dispensing records should maintain consistency with the associated patient and doctor information.

- Where dispensing is recorded, the system should update pharmacy inventory through implemented database or application logic.
- Prescription creation and modification should follow applicable authorization and transaction rules.

1.2.4 Billing and Pharmacy Stock Integration

- Billing transactions should maintain valid references to the corresponding patient accounts and services provided.
- Medicine charges should correspond to medicines and quantities recorded as dispensed, where the billing design supports this relationship.
- Dispensing medicines through the pharmacy should trigger implemented inventory-update logic to reduce stock quantities correctly.
- Billing and inventory operations that must succeed together should use suitable transaction controls to prevent inconsistent records.
- Financial reports should accurately reflect bills, recorded payments, and relevant service details.

1.2.5 Laboratory and Patient History Integration

- Laboratory orders should be linked to valid patient records and prescribing doctors wherever applicable.
- Completed laboratory results should be associated with the corresponding test orders.
- Patient diagnostic histories should support retrieving previous laboratory tests and results.
- Access to laboratory information should follow the implemented user permissions.

1.2.6 Room and Admission Integration

- Each admission should be associated with a valid patient and an available room or bed.
- Bed allocation should be coordinated with occupancy records to prevent conflicting assignments.
- Admission and discharge operations should update occupancy information consistently.
- Room charges should be calculated using the applicable rates and admission duration.
- Patient admission, bed allocation, and related billing updates should use appropriate transaction controls wherever consistency across these operations is required.

1.2.7 Data Consistency and Error Handling

- Database constraints should prevent invalid references, negative pricing, unsupported status values, and other invalid data.
- MySQL should report errors when SQL operations violate implemented constraints.
- The application should handle database errors appropriately and display understandable messages to authorized users.
- Transactions should be used where necessary to maintain consistency across related updates.
- Failed operations should be rolled back when appropriate to prevent partial updates.
- Database relationships and validation rules should be tested to ensure consistent data exchange between hospital modules.

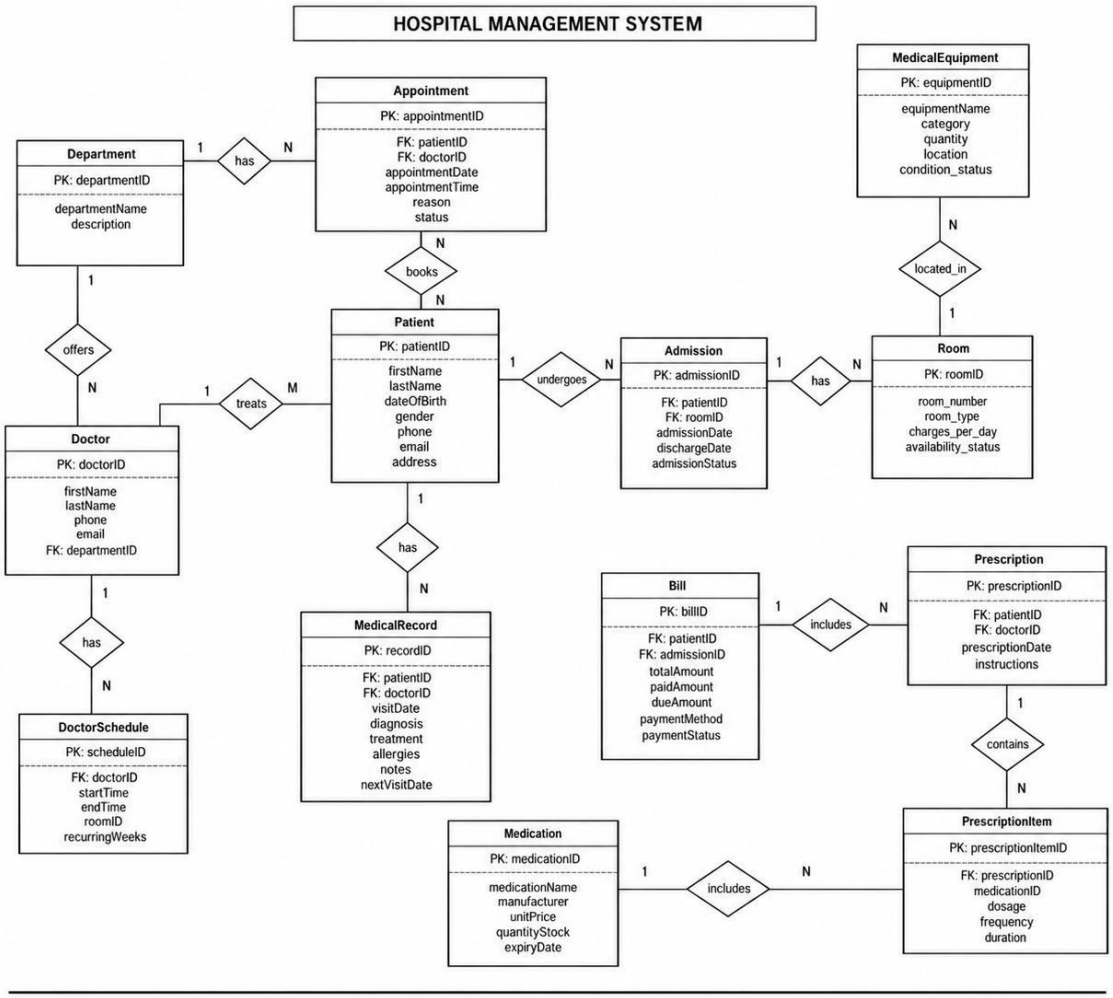
2. ER DIAGRAM OF HOSPITAL MANAGEMENT SYSTEM

The Entity Relationship (ER) Diagram represents the logical structure of the Inventory Management System database. It shows how the 13 entities are connected using primary keys, foreign keys, and relationships.

The database design uses relational concepts to maintain data integrity and organize inventory-related information efficiently.

2.1 Entity Relationship Diagram

3.1 Entity Relationship Diagram



2.2 Entities and Key Identification

The database consists of 13 entities. Their attributes and purposes have already been introduced in Section 1. This section identifies the primary keys, candidate keys, and foreign keys used in the ER design.

Key Notation:

- **PK (Primary Key):** Uniquely identifies each record.
- **FK (Foreign Key):** References the primary key or unique key of a related table.
- **Composite Key:** A key formed using two or more attributes.

Entity	Primary key (PK)	Additional Candidate Key	Foreign Key (FK)
Department	departmentID	DepartmentName	—
Doctor	doctorID	Email	departmentID
DoctorSchedule	scheduleID	—	doctorID
Patient	patientID	Email, Phone	—
Appointment	AppointmentID	—	patientID, doctorID
MedicalRecord	recordID	—	patientID, doctorID
Admission	admissionID	—	patientID, roomID
Room	roomID	room_Number	—
MedicalEquipment	equipmentID	—	roomID
Bill	billID	—	patientID, admissionID
Prescription	prescriptionID	—	patientID, doctorID
PrescriptionItem	prescriptionItemID	—	prescriptionID, medicineID
Medicine	medicineID	medicineName	—

Important Key Characteristics:

1. PrescriptionItem uses a synthetic primary key (prescriptionItemID) while connecting related clinical records using composite foreign keys.
2. The Admission table serves as an operational hub, using foreign keys to connect individual patient files directly with physical hospital rooms.
3. DepartmentID in the Doctor table is mandatory to ensure every physician is strictly assigned to an active operational department.
4. Unique attributes such as room_Number, medicineName, and DepartmentName help prevent duplicate records across administrative fields.

2.3 Relationships and Cardinalities

Cardinality defines the number of records in one entity that can be associated with records in another entity. The Hospital Management System contains 13 direct one-to-many (1:N) relationships and 4 indirect many-to-many (M:N) relationships resolved using junction tables.

2.3.1 Direct One-to-Many Relationships

No.	Related Entities	Connecting Key	Cardinality
1	Department – Doctor	Doctor.departmentID	1:N
2	Department – Appointmt	Appointment.departmentID	1:N
3	Doctor – DoctorSchedule	DoctorSchedule.doctorID	1:N
4	Doctor – Patient	Patient.doctorID	1:N
5	Patient – Appointment	Appointment.patientID	1:N
6	Patient – MedicalRecord	MedicalRecord.patientID	1:N
7	Patient – Admission	Admission.patientID	1:N
8	Room – Admission	Admission.roomID	1:N
9	Room – MedicalEquipment	MedicalEquipment.roomID	1:N
10	Patient – Bill	Bill.patientID	1:N
11	Admission – Bill	Bill.admissionID	1:N
12	Prescription – PrescriptionItem	PrescriptionItem.prescriptionID	1:N
13	Medicine – PrescriptionItem	PrescriptionItem.medicineID	1:N

Explanation of Major Relationships:

1. **Department** – **Doctor:** One department can employ multiple doctors, while each individual doctor belongs to exactly one department.
2. **Doctor** – **DoctorSchedule:** A doctor can have multiple scheduled working hours and shifts recorded, while each schedule entry maps back to one doctor.
3. **Patient** – **Appointment:** A single patient can book multiple appointments over time, but each specific appointment is assigned to one patient.
4. **Room** – **MedicalEquipment**: A hospital room can hold multiple pieces of medical equipment, while each asset is registered to one specific room location.
5. **Admission** – **Bill:** A single patient admission logs hospital stay details that flow directly into a billing statement to track room charges and treatment fees.

2.3.2 Indirect Many-to-Many Relationships

A many-to-many relationship occurs when multiple records of one entity can be associated with multiple records of another entity. These relationships are implemented using junction tables.

No.	Related Entities	Junction Table
1	Patient – Doctor	Appointment / Admission
2	Patient – Medicine	PrescriptionItem
3	Doctor – Medicine	Prescription / PrescriptionItem
4	Patient – Room	Admission

Explanation:

1. Patient – Doctor: A patient can be treated by multiple doctors during their care, and a doctor treats multiple patients. This relationship is dynamically resolved through Appointment and Admission records.
2. Patient – Medicine: A patient can receive multiple medications over time, and a single type of medicine can be prescribed to many different patients. The PrescriptionItem table acts as the bridge resolving this mapping.
3. Doctor – Medicine: A doctor can prescribe various medications, and a specific medication can be ordered by multiple doctors across the hospital. This is tracked through the unified Prescription system.
4. Patient – Room: A patient can stay in different rooms over multiple hospital visits, and a single hospital room hosts different patients over time. The Admission entity this timeline seamlessly.

3. DATABASE IMPLEMENTATION

Database implementation is the process of creating a database and its tables using SQL commands. In the Hospital Management System, MySQL is used to store and manage patient details, doctor information, appointments, medical records, prescriptions, room information, admissions, billing details, medicines, and medical equipment. The database helps maintain accurate records, reduce data duplication, and retrieve hospital information efficiently.

3.1 Database Creation

The database `HospitalManagementDB` is created using the `CREATE DATABASE` command. The `USE` command selects the database for further operations.

SQL Command:

```
mysql> create database HospitalManagement;
Query OK, 1 row affected (0.04 sec)

mysql> use HospitalManagement;
Database changed
mysql> CREATE TABLE Department (
    ->     DepartmentID INT AUTO_INCREMENT PRIMARY KEY,
    ->     DepartmentName VARCHAR(100) NOT NULL,
    ->     Description TEXT
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
```

Figure 4.1: Creation and selection of the Hospital Management database.

3.2 Table Creation

Thirteen tables are created in the Hospital Management database to store information required for hospital operations. Primary keys are used to uniquely identify records, while foreign keys establish relationships between related tables. These tables help manage patient registration, doctor schedules, appointments, medical records, prescriptions, medicines, rooms, admissions, and billing.

3.2.1 Department Table

The Department table stores information about the different departments available in the hospital, such as Cardiology, Neurology, Orthopedics, and General Medicine. It includes the department ID, department name, and description.

```
mysql> INSERT INTO Department (DepartmentName, Description)
    -> VALUES
    -> ('Cardiology', 'Heart related treatments'),
    -> ('Neurology', 'Brain and nervous system'),
    -> ('Orthopedics', 'Bones and joints');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Department;
+--------------+-----------------+--------------------------+
| DepartmentID | DepartmentName  | Description              |
+--------------+-----------------+--------------------------+
|            1 | Cardiology      | Heart related treatments |
|            2 | Neurology       | Brain and nervous system |
|            3 | Orthopedics     | Bone and joint treatments |
|            4 | Pediatrics      | Child healthcare         |
|            5 | General Medicine | General health issues    |
|            6 | Cardiology      | Heart related treatments |
|            7 | Neurology       | Brain and nervous system |
|            8 | Orthopedics     | Bones and joints         |
+--------------+-----------------+--------------------------+
8 rows in set (0.00 sec)
```

Figure 4.2: Departments table and its records.

3.2.2 Doctor Table

The Doctor table stores details of doctors working in the hospital, including their names, specializations, phone numbers, email addresses, and department information. Each doctor is assigned a unique DoctorID.

```
mysql> CREATE TABLE Doctor (
    ->     DoctorID INT AUTO_INCREMENT PRIMARY KEY,
    ->     DoctorName VARCHAR(100) NOT NULL,
    ->     Specialization VARCHAR(100),
    ->     Phone VARCHAR(15),
    ->     Email VARCHAR(100),
    ->     DepartmentID INT,
    ->     FOREIGN KEY (DepartmentID)
    ->         REFERENCES Department(DepartmentID)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO Doctor (DoctorName, Specialization, Phone, Email, DepartmentID)
    -> VALUES
    -> ('Dr. Rajesh Kumar','Cardiologist','9876543210','rajesh@gmail.com',1),
    -> ('Dr. Priya Sharma','Neurologist','9876543211','priya@gmail.com',2),
    -> ('Dr. Amit Singh','Orthopedic','9876543212','amit@gmail.com',3);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Doctor;
+----------+-----------------+----------------+------------+------------------+--------------+
| DoctorID | DoctorName      | Specialization | Phone      | Email            | DepartmentID |
+----------+-----------------+----------------+------------+------------------+--------------+
|        1 | Dr. Rajesh Kumar | Cardiologist   | 9876543210 | rajesh@gmail.com |            1 |
|        2 | Dr. Priya Sharma | Neurologist    | 9876543211 | priya@gmail.com  |            2 |
|        3 | Dr. Amit Singh   | Orthopedic     | 9876543212 | amit@gmail.com   |            3 |
+----------+-----------------+----------------+------------+------------------+--------------+
3 rows in set (0.00 sec)
```

Figure 4.3: Doctor table and its records.

3.2.3 Patient Table

The Patient table stores personal and contact information of patients registered in the hospital. It includes the patient ID, name, date of birth, gender, phone number, email address, and residential address.

```
mysql> CREATE TABLE Patient (
    ->     PatientID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientName VARCHAR(100) NOT NULL,
    ->     DateOfBirth DATE,
    ->     Gender VARCHAR(20),
    ->     Phone VARCHAR(15),
    ->     Email VARCHAR(100),
    ->     Address VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Patient (PatientName, DateOfBirth, Gender, Phone, Email, Address)
    -> VALUES
    -> ('Rohan Mehta','2001-05-15','Male','9123456789','rohan@gmail.com','Hyderabad'),
    -> ('Sneha Rao','1998-09-20','Female','9234567890','sneha@gmail.com','Bangalore'),
    -> ('Karan Patel','2003-02-11','Male','9345678901','karan@gmail.com','Mumbai');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Patient;
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
| PatientID | PatientName | DateOfBirth | Gender | Phone      | Email           | Address   |
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
|         1 | Rohan Mehta | 2001-05-15  | Male   | 9123456789 | rohan@gmail.com | Hyderabad |
|         2 | Sneha Rao   | 1998-09-20  | Female | 9234567890 | sneha@gmail.com | Bangalore |
|         3 | Karan Patel | 2003-02-11  | Male   | 9345678901 | karan@gmail.com | Mumbai    |
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
3 rows in set (0.00 sec)
```

Figure 4.4: Patient table and its records.

3.2.4 DoctorSchedule Table

The DoctorSchedule table maintains doctors' working days, starting times, ending times, and availability status. It helps the hospital manage doctor availability and schedule patient appointments efficiently.

```
mysql> CREATE TABLE DoctorSchedule (
    ->     ScheduleID INT AUTO_INCREMENT PRIMARY KEY,
    ->     DoctorID INT,
    ->     WorkingDay VARCHAR(20),
    ->     StartTime TIME,
    ->     EndTime TIME,
    ->     AvailabilityStatus VARCHAR(20),
    ->     FOREIGN KEY (DoctorID)
    ->         REFERENCES Doctor(DoctorID)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO DoctorSchedule (DoctorID, WorkingDay, StartTime, EndTime, AvailabilityStatus)
    -> VALUES
    -> (1,'Monday','09:00:00','17:00:00','Available'),
    -> (2,'Tuesday','10:00:00','16:00:00','Available'),
    -> (3,'Wednesday','08:00:00','14:00:00','Busy');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM DoctorSchedule;
+------------+----------+------------+-----------+----------+--------------------+
| ScheduleID | DoctorID | WorkingDay | StartTime | EndTime  | AvailabilityStatus |
+------------+----------+------------+-----------+----------+--------------------+
|          1 |        1 | Monday     | 09:00:00  | 17:00:00 | Available          |
|          2 |        2 | Tuesday    | 10:00:00  | 16:00:00 | Available          |
|          3 |        3 | Wednesday  | 08:00:00  | 14:00:00 | Busy               |
+------------+----------+------------+-----------+----------+--------------------+
3 rows in set (0.00 sec)
```

Figure 4.5: DoctorSchedule table and its records.

3.2.5 Appointment Table

The Appointment table records appointments scheduled between patients and doctors. It stores the appointment date, time, reason for consultation, and appointment status.

```
mysql> CREATE TABLE Appointment (
    ->     AppointmentID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientID INT,
    ->     DoctorID INT,
    ->     AppointmentDate DATE,
    ->     AppointmentTime TIME,
    ->     Reason TEXT,
    ->     Status VARCHAR(30),
    ->     FOREIGN KEY (PatientID)
    ->         REFERENCES Patient(PatientID),
    ->     FOREIGN KEY (DoctorID)
    ->         REFERENCES Doctor(DoctorID)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> INSERT INTO Appointment (PatientID, DoctorID, AppointmentDate, AppointmentTime, Reason, Status)
    -> VALUES
    -> (1,1,'2026-10-20','10:00:00','Chest Pain','Confirmed'),
    -> (2,2,'2026-10-21','11:30:00','Migraine','Pending'),
    -> (3,3,'2026-10-22','09:15:00','Fracture Checkup','Completed');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Appointment;
+---------------+-----------+----------+-----------------+-----------------+------------------+-----------+
| AppointmentID | PatientID | DoctorID | AppointmentDate | AppointmentTime | Reason           | Status    |
+---------------+-----------+----------+-----------------+-----------------+------------------+-----------+
|             1 |         1 |        1 | 2026-10-20      | 10:00:00        | Chest Pain       | Confirmed |
|             2 |         2 |        2 | 2026-10-21      | 11:30:00        | Migraine         | Pending   |
|             3 |         3 |        3 | 2026-10-22      | 09:15:00        | Fracture Checkup | Completed |
+---------------+-----------+----------+-----------------+-----------------+------------------+-----------+
3 rows in set (0.00 sec)
```

Figure 4.6: Appointment table and its records.

3.2.6 MedicalRecord Table

The MedicalRecord table maintains patients' medical histories, diagnoses, symptoms, treatments, allergies, and record dates. It helps doctors review previous medical information before providing treatment.

```
mysql> CREATE TABLE MedicalRecord (
    ->     RecordID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientID INT,
    ->     DoctorID INT,
    ->     Diagnosis TEXT,
    ->     Symptoms TEXT,
    ->     Treatment TEXT,
    ->     Allergies TEXT,
    ->     RecordDate DATE,
    ->     FOREIGN KEY (PatientID)
    ->         REFERENCES Patient(PatientID),
    ->     FOREIGN KEY (DoctorID)
    ->         REFERENCES Doctor(DoctorID)
    -> );
Query OK, 0 rows affected (0.13 sec)
```

```
mysql> INSERT INTO MedicalRecord (PatientID, DoctorID, Diagnosis, Symptoms, Treatment, Allergies, RecordDate)
    -> VALUES
    -> (1,1,'Heart Disease','Chest Pain','Medication','None','2026-10-20'),
    -> (2,2,'Migraine','Headache','Painkillers','Penicillin','2026-10-21'),
    -> (3,3,'Leg Fracture','Pain','Cast','None','2026-10-22');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM MedicalRecord;
+----------+-----------+----------+---------------+------------+------------+------------+------------+
| RecordID | PatientID | DoctorID | Diagnosis     | Symptoms   | Treatment  | Allergies  | RecordDate |
+----------+-----------+----------+---------------+------------+------------+------------+------------+
|        1 |         1 |        1 | Heart Disease | Chest Pain | Medication | None       | 2026-10-20 |
|        2 |         2 |        2 | Migraine      | Headache   | Painkillers| Penicillin | 2026-10-21 |
|        3 |         3 |        3 | Leg Fracture  | Pain       | Cast       | None       | 2026-10-22 |
+----------+-----------+----------+---------------+------------+------------+------------+------------+
3 rows in set (0.00 sec)
```

Figure 4.7: MedicalRecord table and its records.

3.2.7 Prescription Table

The Prescription table stores prescriptions issued by doctors to patients. It includes prescription details, the associated patient and doctor, the medical record, the prescription date, and instructions.

```
mysql> CREATE TABLE Prescription (
    ->     PrescriptionID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientID INT,
    ->     DoctorID INT,
    ->     RecordID INT,
    ->     PrescriptionDate DATE,
    ->     Instructions TEXT,
    ->     FOREIGN KEY (PatientID)
    ->         REFERENCES Patient(PatientID),
    ->     FOREIGN KEY (DoctorID)
    ->         REFERENCES Doctor(DoctorID),
    ->     FOREIGN KEY (RecordID)
    ->         REFERENCES MedicalRecord(RecordID)
    -> );
Query OK, 0 rows affected (0.17 sec)
```

```
mysql> INSERT INTO Prescription (PatientID, DoctorID, RecordID, PrescriptionDate, Instructions)
    -> VALUES
    -> (1,1,1,'2026-10-20','Take medicines after food'),
    -> (2,2,2,'2026-10-21','Sleep well and avoid stress'),
    -> (3,3,3,'2026-10-22','Wear cast for 6 weeks');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Prescription;
+----------------+-----------+----------+----------+------------------+----------------------------+
| PrescriptionID | PatientID | DoctorID | RecordID | PrescriptionDate | Instructions               |
+----------------+-----------+----------+----------+------------------+----------------------------+
|              1 |         1 |        1 |        1 | 2026-10-20       | Take medicines after food  |
|              2 |         2 |        2 |        2 | 2026-10-21       | Sleep well and avoid stress |
|              3 |         3 |        3 |        3 | 2026-10-22       | Wear cast for 6 weeks      |
+----------------+-----------+----------+----------+------------------+----------------------------+
3 rows in set (0.00 sec)
```

Figure 4.8: Prescription table and its records.

3.2.8 Medicine Table

The Medicine table maintains information about medicines available in the hospital pharmacy. It includes medicine names, manufacturers, unit prices, stock quantities, reorder levels, and expiry dates.

```
mysql> CREATE TABLE Medicine (
    ->     MedicineID INT AUTO_INCREMENT PRIMARY KEY,
    ->     MedicineName VARCHAR(100),
    ->     Manufacturer VARCHAR(100),
    ->     UnitPrice DECIMAL(10,2),
    ->     QuantityInStock INT,
    ->     ReorderLevel INT,
    ->     ExpiryDate DATE
    -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Medicine (MedicineName, Manufacturer, UnitPrice, QuantityInStock, ReorderLevel, ExpiryDate)
    -> VALUES
    -> ('Paracetamol','Cipla',10.50,500,100,'2028-01-01'),
    -> ('Aspirin','Sun Pharma',15.75,300,50,'2027-12-31'),
    -> ('Amoxicillin','Abbott',25.00,200,30,'2027-10-10');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Medicine;
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
| MedicineID | MedicineName | Manufacturer | UnitPrice | QuantityInStock | ReorderLevel | ExpiryDate |
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
|          1 | Paracetamol  | Cipla        |     10.50 |             500 |          100 | 2028-01-01 |
|          2 | Aspirin      | Sun Pharma   |     15.75 |             300 |           50 | 2027-12-31 |
|          3 | Amoxicillin  | Abbott       |     25.00 |             200 |           30 | 2027-10-10 |
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
3 rows in set (0.00 sec)
```

Figure 4.9: Medicine table and its records.

3.2.9 PrescriptionItem Table

The PrescriptionItem table stores individual medicines prescribed to patients. It includes the medicine ID, dosage, frequency, and duration of each prescribed medicine.

```
mysql> CREATE TABLE PrescriptionItem (
    ->     PrescriptionItemID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PrescriptionID INT,
    ->     MedicineID INT,
    ->     Dosage VARCHAR(50),
    ->     Frequency VARCHAR(50),
    ->     Duration VARCHAR(50),
    ->     FOREIGN KEY (PrescriptionID)
    ->         REFERENCES Prescription(PrescriptionID),
    ->     FOREIGN KEY (MedicineID)
    ->         REFERENCES Medicine(MedicineID)
    -> );
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> INSERT INTO PrescriptionItem (PrescriptionID, MedicineID, Dosage, Frequency, Duration)
    -> VALUES
    -> (1,1,'500mg','Twice a day','5 Days'),
    -> (2,2,'250mg','Once a day','7 Days'),
    -> (3,3,'500mg','Three times a day','10 Days');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM PrescriptionItem;
+--------------------+----------------+------------+--------+-------------------+----------+
| PrescriptionItemID | PrescriptionID | MedicineID | Dosage | Frequency         | Duration |
+--------------------+----------------+------------+--------+-------------------+----------+
|                  1 |              1 |          1 | 500mg  | Twice a day       | 5 Days   |
|                  2 |              2 |          2 | 250mg  | Once a day        | 7 Days   |
|                  3 |              3 |          3 | 500mg  | Three times a day | 10 Days  |
+--------------------+----------------+------------+--------+-------------------+----------+
3 rows in set (0.00 sec)
```

Figure 4.10: PrescriptionItem table and its records.

3.2.10 MedicalEquipment Table

The MedicalEquipment table stores information about medical equipment used in the hospital, including equipment names, categories, quantities, unit prices, and condition status. It helps track equipment availability and maintenance requirements.

```
mysql> CREATE TABLE MedicalEquipment (
    ->     EquipmentID INT AUTO_INCREMENT PRIMARY KEY,
    ->     EquipmentName VARCHAR(100),
    ->     Category VARCHAR(100),
    ->     Quantity INT,
    ->     UnitPrice DECIMAL(10,2),
    ->     ConditionStatus VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO MedicalEquipment (EquipmentName, Category, Quantity, UnitPrice, ConditionStatus)
    -> VALUES
    -> ('ECG Machine','Diagnostic',5,75000.00,'Good'),
    -> ('Wheelchair','Support',20,8000.00,'Excellent'),
    -> ('X-Ray Machine','Imaging',2,500000.00,'Good');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM MedicalEquipment;
+-------------+---------------+------------+----------+-----------+-----------------+
| EquipmentID | EquipmentName | Category   | Quantity | UnitPrice | ConditionStatus |
+-------------+---------------+------------+----------+-----------+-----------------+
|           1 | ECG Machine   | Diagnostic |        5 |  75000.00 | Good            |
|           2 | Wheelchair    | Support    |       20 |   8000.00 | Excellent       |
|           3 | X-Ray Machine | Imaging    |        2 | 500000.00 | Good            |
+-------------+---------------+------------+----------+-----------+-----------------+
3 rows in set (0.00 sec)
```

Figure 4.11: MedicalEquipment table and its records.

3.2.11 Room Table

The Room table maintains information about hospital rooms, including room numbers, room types, daily charges, and availability status. It helps the hospital assign suitable rooms to admitted patients.

```
mysql> CREATE TABLE Room (
    ->     RoomID INT AUTO_INCREMENT PRIMARY KEY,
    ->     RoomNumber VARCHAR(20),
    ->     RoomType VARCHAR(50),
    ->     ChargesPerDay DECIMAL(10,2),
    ->     AvailabilityStatus VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Room (RoomNumber, RoomType, ChargesPerDay, AvailabilityStatus)
    -> VALUES
    -> ('101','General',1000.00,'Available'),
    -> ('201','ICU',5000.00,'Occupied'),
    -> ('301','Private',3000.00,'Available');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Room;
+--------+------------+----------+---------------+--------------------+
| RoomID | RoomNumber | RoomType | ChargesPerDay | AvailabilityStatus |
+--------+------------+----------+---------------+--------------------+
|      1 | 101        | General  |       1000.00 | Available          |
|      2 | 201        | ICU      |       5000.00 | Occupied           |
|      3 | 301        | Private  |       3000.00 | Available          |
+--------+------------+----------+---------------+--------------------+
3 rows in set (0.00 sec)
```

Figure 4.12: Room table and its records.

3.2.12 Admission Table

The Admission table records patient admissions, room assignments, admission dates, discharge dates, and admission status. It helps manage inpatient information and room occupancy.

```
mysql> CREATE TABLE Admission (
    ->     AdmissionID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientID INT,
    ->     RoomID INT,
    ->     AdmissionDate DATE,
    ->     DischargeDate DATE,
    ->     AdmissionStatus VARCHAR(30),
    ->     FOREIGN KEY (PatientID)
    ->         REFERENCES Patient(PatientID),
    ->     FOREIGN KEY (RoomID)
    ->         REFERENCES Room(RoomID)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO Admission (PatientID, RoomID, AdmissionDate, DischargeDate, AdmissionStatus)
    -> VALUES
    -> (1,2,'2026-10-20','2026-10-25','Discharged'),
    -> (2,1,'2026-10-21',NULL,'Admitted'),
    -> (3,3,'2026-10-22','2026-10-30','Discharged');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Admission;
+-------------+-----------+--------+---------------+---------------+-----------------+
| AdmissionID | PatientID | RoomID | AdmissionDate | DischargeDate | AdmissionStatus |
+-------------+-----------+--------+---------------+---------------+-----------------+
|           1 |         1 |      2 | 2026-10-20    | 2026-10-25    | Discharged      |
|           2 |         2 |      1 | 2026-10-21    | NULL          | Admitted        |
|           3 |         3 |      3 | 2026-10-22    | 2026-10-30    | Discharged      |
+-------------+-----------+--------+---------------+---------------+-----------------+
3 rows in set (0.00 sec)
```

Figure 4.13: Admission table and its records.

3.2.13 Bill Table

The Bill table stores billing information for patients, including consultation charges, treatment charges, other charges, total amounts, payment methods, and payment status. It helps maintain hospital financial records.

```
mysql> CREATE TABLE Bill (
    ->     BillID INT AUTO_INCREMENT PRIMARY KEY,
    ->     PatientID INT,
    ->     AdmissionID INT,
    ->     BillDate DATE,
    ->     ConsultationCharges DECIMAL(10,2),
    ->     TreatmentCharges DECIMAL(10,2),
    ->     OtherCharges DECIMAL(10,2),
    ->     TotalAmount DECIMAL(12,2),
    ->     PaymentMethod VARCHAR(30),
    ->     PaymentStatus VARCHAR(30),
    ->     FOREIGN KEY (PatientID)
    ->         REFERENCES Patient(PatientID),
    ->     FOREIGN KEY (AdmissionID)
    ->         REFERENCES Admission(AdmissionID)
    -> );
Query OK, 0 rows affected (0.09 sec)
```

```
mysql> INSERT INTO Bill (PatientID, AdmissionID, BillDate, ConsultationCharges, TreatmentCharges, OtherCharges, TotalAmount, PaymentMethod, PaymentStatus)
    -> VALUES
    -> (1,1,'2026-10-25',500.00,4500.00,1000.00,6000.00,'Cash','Paid'),
    -> (2,2,'2026-10-23',600.00,2500.00,500.00,3600.00,'Card','Pending'),
    -> (3,3,'2026-10-30',700.00,5500.00,800.00,7000.00,'UPI','Paid');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Bill;
+--------+-----------+-------------+------------+--------------------+-----------------+-------------+-------------+---------------+---------------+
| BillID | PatientID | AdmissionID | BillDate   | ConsultationCharges | TreatmentCharges | OtherCharges | TotalAmount | PaymentMethod | PaymentStatus |
+--------+-----------+-------------+------------+--------------------+-----------------+-------------+-------------+---------------+---------------+
|      1 |         1 |           1 | 2026-10-25 |             500.00 |         4500.00 |      1000.00 |     6000.00 | Cash          | Paid          |
|      2 |         2 |           2 | 2026-10-23 |             600.00 |         2500.00 |       500.00 |     3600.00 | Card          | Pending       |
|      3 |         3 |           3 | 2026-10-30 |             700.00 |         5500.00 |       800.00 |     7000.00 | UPI           | Paid          |
+--------+-----------+-------------+------------+--------------------+-----------------+-------------+-------------+---------------+---------------+
3 rows in set (0.00 sec)

mysql>
```

Figure 4.14: Bill table and its records.

4. SQL QUERIES and OUTPUTS

An **SQL query** is a declarative statement that specifies a desired data outcome rather than the procedural steps required to achieve it. The user defines the criteria for data selection, modification, or deletion, leaving the physical execution strategy entirely to the database management system's internal optimizer.

5.1.1 UPDATE Operation

- **Query/operation description:** Modify an existing patient's phone number in the Patient table.

SQL code:

- **UPDATE Patient SET Phone = ‘ 9111222333 ‘ WHERE PatientID = 1;**

```
mysql> UPDATE Patient
    -> SET Phone = '9111222333'
    -> WHERE PatientID = 1;
Query OK, 1 row affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from Patient;
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
| PatientID | PatientName | DateOfBirth | Gender | Phone      | Email           | Address   |
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
|         1 | Rohan Mehta | 2001-05-15  | Male   | 9111222333 | rohan@gmail.com | Hyderabad |
|         2 | Sneha Rao   | 1998-09-20  | Female | 9234567890 | sneha@gmail.com | Bangalore |
|         3 | Karan Patel | 2003-02-11  | Male   | 9345678901 | karan@gmail.com | Mumbai    |
+-----------+-------------+-------------+--------+------------+-----------------+-----------+
3 rows in set (0.00 sec)
```

5.1.2 DELETE Operation

- **Query/operation description:** Remove a specific appointment from the Appointment table.

SQL code:

- **DELETE FROM Appointment WHERE AppointmentID = 3;**

```
mysql> DELETE FROM Appointment
    -> WHERE AppointmentID = 3;
Query OK, 1 row affected (0.04 sec)

mysql> select * from Appointment;
+---------------+-----------+----------+-----------------+-----------------+------------+-----------+
| AppointmentID | PatientID | DoctorID | AppointmentDate | AppointmentTime | Reason     | Status    |
+---------------+-----------+----------+-----------------+-----------------+------------+-----------+
|             1 |         1 |        1 | 2026-10-20      | 10:00:00        | Chest Pain | Confirmed |
|             2 |         2 |        2 | 2026-10-21      | 11:30:00        | Migraine   | Pending   |
+---------------+-----------+----------+-----------------+-----------------+------------+-----------+
2 rows in set (0.00 sec)
```

5.1.3 JOIN, AS, & ORDER BY

- **Query/operation description:** Retrieve patient names and room charges using an inner join and aliases (AS), ordered from highest to lowest charges.

SQL code:

- SELECT p.PatientName AS FullName, r.RoomNumber, r.ChargesPerDay FROM Admission ad JOIN Patient p ON ad.PatientID = p.PatientID JOIN Room r ON
  ad.RoomID = r.RoomID ORDER BY r.ChargesPerDay DESC;

```
mysql> SELECT p.PatientName AS FullName, r.RoomNumber, r.ChargesPerDay
    -> FROM Admission ad
    -> JOIN Patient p ON ad.PatientID = p.PatientID
    -> JOIN Room r ON ad.RoomID = r.RoomID
    -> ORDER BY r.ChargesPerDay DESC;
+------------+------------+---------------+
| FullName   | RoomNumber | ChargesPerDay |
+------------+------------+---------------+
| Rohan Mehta | 201        |       5000.00 |
| Karan Patel | 301        |       3000.00 |
| Sneha Rao   | 101        |       1000.00 |
+------------+------------+---------------+
3 rows in set (0.04 sec)
```

5.1.4 GROUP BY & HAVING

- **Query/operation description:** Find departments that have more than 1 doctor assigned.

SQL code:

- SELECT dept.DepartmentName, COUNT(d.DoctorID) AS TotalDoctors FROM Department dept JOIN Doctor d ON dept.DepartmentID = d.DepartmentID GROUP BY dept.DepartmentName HAVING COUNT(d.DoctorID) > 1;

```
mysql> SELECT dept.DepartmentName, COUNT(d.DoctorID) AS TotalDoctors
    -> FROM Department dept
    -> JOIN Doctor d ON dept.DepartmentID = d.DepartmentID
    -> GROUP BY dept.DepartmentName
    -> HAVING COUNT(d.DoctorID) > 1;
Empty set (0.02 sec)
```

5.1.5 UNION

- **Query/operation description:** Combine patient names and doctor names into a single unified directory list.

SQL code:

- SELECT PatientName AS PersonName, 'Patient' AS Role FROM Patient UNION SELECT DoctorName AS PersonName, 'Doctor' AS Role FROM Doctor;

```
mysql> SELECT PatientName AS PersonName, 'Patient' AS Role FROM Patient
    -> UNION
    -> SELECT DoctorName AS PersonName, 'Doctor' AS Role FROM Doctor;
+------------------+---------+
| PersonName       | Role    |
+------------------+---------+
| Rohan Mehta      | Patient |
| Sneha Rao        | Patient |
| Karan Patel      | Patient |
| Dr. Rajesh Kumar | Doctor  |
| Dr. Priya Sharma | Doctor  |
| Dr. Amit Singh   | Doctor  |
+------------------+---------+
6 rows in set (0.04 sec)
```

5.1.6 EXISTS

- **Query/operation description:** Find patients who have at least one billing record marked as 'Paid'.

SQL code:

- SELECT p.PatientName FROM Patient p WHERE EXISTS (SELECT 1 FROM Bill b WHERE b.PatientID = p.PatientID AND b.PaymentStatus = 'Paid');

```
mysql> SELECT p.PatientName
    -> FROM Patient p
    -> WHERE EXISTS (
    ->     SELECT 1 FROM Bill b
    ->     WHERE b.PatientID = p.PatientID AND b.PaymentStatus = 'Paid'
    -> );
+-------------+
| PatientName |
+-------------+
| Rohan Mehta |
| Karan Patel |
+-------------+
2 rows in set (0.04 sec)
```

5.1.7 ANY & ALL

- **Query/operation description:** Find medicines whose unit price is greater than *any* medicine price from 'Cipla'.

SQL code:

- SELECT MedicineName, UnitPrice FROM Medicine WHERE UnitPrice > ANY (SELECT UnitPrice FROM Medicine WHERE Manufacturer = 'Cipla');

```
mysql> SELECT MedicineName, UnitPrice
    -> FROM Medicine
    -> WHERE UnitPrice > ANY (
    ->     SELECT UnitPrice FROM Medicine WHERE Manufacturer = 'Cipla'
    -> );
+--------------+-----------+
| MedicineName | UnitPrice |
+--------------+-----------+
| Aspirin      |     15.75 |
| Amoxicillin  |     25.00 |
+--------------+-----------+
2 rows in set (0.05 sec)
```

5.1.8 Aggregates, DISTINCT, BETWEEN, & IS NULL

- **Query/operation description:** Calculate total revenue and average treatment costs where bills fall within date ranges and discharge dates are not null.

SQL code:

- SELECT COUNT(DISTINCT PaymentMethod) AS UniquePaymentMethods, SUM(TotalAmount) AS TotalRevenue, AVG(TreatmentCharges) AS AvgTreatmentCost FROM Bill b JOIN Admission ad ON b.AdmissionID = ad.AdmissionID WHERE b.BillDate BETWEEN '2026-10-01' AND '2026-10-31' AND ad.DischargeDate IS NOT NULL;

```
mysql> SELECT
    ->     COUNT(DISTINCT PaymentMethod) AS UniquePaymentMethods,
    ->     SUM(TotalAmount) AS TotalRevenue,
    ->     AVG(TreatmentCharges) AS AvgTreatmentCost
    -> FROM Bill b
    -> JOIN Admission ad ON b.AdmissionID = ad.AdmissionID
    -> WHERE b.BillDate BETWEEN '2026-10-01' AND '2026-10-31'
    ->   AND ad.DischargeDate IS NOT NULL;
+----------------------+--------------+------------------+
| UniquePaymentMethods | TotalRevenue | AvgTreatmentCost |
+----------------------+--------------+------------------+
|                    2 |     13600.00 |      5000.000000 |
+----------------------+--------------+------------------+
1 row in set (0.01 sec)
```

5.1.9 Wildcards & Advanced String Functions (CONCAT, SUBSTRING, LENGTH, UPPER, LOWER)

- **Query/operation description:** Format doctor details using string manipulations and wildcards to find names starting with 'Dr.'.

SQL code:

- SELECT UPPER(DoctorName) AS UppercaseName, LOWER(Specialization) AS LowerSpecialization, CONCAT('ID: ', DoctorID, ' - ', Specialization) AS DoctorSummary, SUBSTRING(Email, 1, 5) AS EmailPrefix, LENGTH(Phone) AS PhoneLength FROM Doctor WHERE DoctorName LIKE 'Dr.%';

```
mysql> SELECT
    ->     UPPER(DoctorName) AS UppercaseName,
    ->     LOWER(Specialization) AS LowerSpecialization,
    ->     CONCAT('ID: ', DoctorID, ' - ', Specialization) AS DoctorSummary,
    ->     SUBSTRING(Email, 1, 5) AS EmailPrefix,
    ->     LENGTH(Phone) AS PhoneLength
    -> FROM Doctor
    -> WHERE DoctorName LIKE 'Dr.%';
+-----------------+--------------------+---------------------+-------------+-------------+
| UppercaseName   | LowerSpecialization | DoctorSummary       | EmailPrefix | PhoneLength |
+-----------------+--------------------+---------------------+-------------+-------------+
| DR. RAJESH KUMAR | cardiologist       | ID: 1 - Cardiologist | rajes       |          10 |
| DR. PRIYA SHARMA | neurologist        | ID: 2 - Neurologist  | priya       |          10 |
| DR. AMIT SINGH   | orthopedic         | ID: 3 - Orthopedic   | amit@       |          10 |
+-----------------+--------------------+---------------------+-------------+-------------+
3 rows in set (0.06 sec)
```

5.1.10 LIMIT & OFFSET

- **Query/operation description:** Retrieve 2 patients starting from the second record for pagination.

SQL code:

- SELECT PatientID, PatientName, Address FROM Patient LIMIT 2 OFFSET 1;

```
mysql> SELECT PatientID, PatientName, Address
    -> FROM Patient
    -> LIMIT 2 OFFSET 1;
+-----------+-------------+-----------+
| PatientID | PatientName | Address   |
+-----------+-------------+-----------+
|         2 | Sneha Rao   | Bangalore |
|         3 | Karan Patel | Mumbai    |
+-----------+-------------+-----------+
2 rows in set (0.00 sec)
```

5.1.11. Stored Procedure

- **Query/operation description:** Create a reusable procedure to fetch medical history for a given patient ID.

SQL code:

- DELIMITER //
- CREATE PROCEDURE GetPatientMedicalHistory(IN p_PatientID INT)
- BEGIN

- SELECT mr.RecordID, mr.Diagnosis, mr.Treatment
- FROM MedicalRecord mr
- WHERE mr.PatientID = p_PatientID;
- END//
- DELIMITER;
- CALL GetPatientMedicalHistory(1);

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetPatientMedicalHistory(IN p_PatientID INT)
    -> BEGIN
    ->     SELECT mr.RecordID, mr.Diagnosis, mr.Treatment
    ->     FROM MedicalRecord mr
    ->     WHERE mr.PatientID = p_PatientID;
    -> END //
Query OK, 0 rows affected (0.09 sec)

mysql> DELIMITER ;
mysql> CALL GetPatientMedicalHistory(1);
+----------+---------------+------------+
| RecordID | Diagnosis     | Treatment  |
+----------+---------------+------------+
|        1 | Heart Disease | Medication |
+----------+---------------+------------+
1 row in set (0.05 sec)

Query OK, 0 rows affected (0.05 sec)
```

5.1.12. Trigger

- **Query/operation description:** Create a trigger to sanitize unit prices before insertion into the Medicine table.
- DELIMITER //
- CREATE TRIGGER BeforeMedicineInsert
- BEFORE INSERT ON Medicine
- FOR EACH ROW
- BEGIN
- IF NEW.UnitPrice < 0 THEN
- SET NEW.UnitPrice = 0;
- END IF;

- END //
- DELIMITER ;
- INSERT INTO Medicine (MedicineName, Manufacturer, UnitPrice, QuantityInStock, ReorderLevel, ExpiryDate) VALUES ('TestMedicine', 'TestLab', - 15.00, 100, 20, '2028-01-01');
- SELECT * FROM Medicine WHERE MedicineName = 'TestMedicine';

```
mysql> DELIMITER //
mysql> CREATE TRIGGER BeforeMedicineInsert
    -> BEFORE INSERT ON Medicine
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.UnitPrice < 0 THEN
    ->         SET NEW.UnitPrice = 0;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.06 sec)

mysql> DELIMITER ;
mysql> INSERT INTO Medicine (MedicineName, Manufacturer, UnitPrice, QuantityInStock, ReorderLevel, ExpiryDate)
    -> VALUES ('TestMedicine', 'TestLab', -15.00, 100, 20, '2028-01-01');
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM Medicine WHERE MedicineName = 'TestMedicine';
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
| MedicineID | MedicineName | Manufacturer | UnitPrice | QuantityInStock | ReorderLevel | ExpiryDate |
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
|          4 | TestMedicine | TestLab      |      0.00 |             100 |           20 | 2028-01-01 |
+------------+--------------+--------------+-----------+-----------------+--------------+------------+
1 row in set (0.00 sec)
```

5.1.13. INNER JOIN (Doctor & Department Mapping)

- **Query/operation description:** List each doctor's name along with their respective department name and specialization.

SQL code:

- SELECT d.DoctorName, dept.DepartmentName, d.Specialization FROM Doctor d JOIN Department dept ON d.DepartmentID = dept.DepartmentID;

```
mysql> SELECT d.DoctorName, dept.DepartmentName, d.Specialization
    -> FROM Doctor d
    -> JOIN Department dept ON d.DepartmentID = dept.DepartmentID;
+------------------+----------------+----------------+
| DoctorName       | DepartmentName | Specialization |
+------------------+----------------+----------------+
| Dr. Rajesh Kumar | Cardiology     | Cardiologist   |
| Dr. Priya Sharma | Neurology      | Neurologist    |
| Dr. Amit Singh   | Orthopedics    | Orthopedic     |
+------------------+----------------+----------------+
3 rows in set (0.00 sec)
```

5.1.14. MULTI-TABLE JOIN (Patient Appointments & Doctors)

- **Query/operation description:** Retrieve patient appointment details including doctor names, appointment reason, and status.

SQL code:

- SELECT p.Patient Name, an.Appointment Date, a.Reason, doc.Doctor Name, a.Status FROM Appointment a JOIN Patient p ON a.PatientID = p.PatientID JOIN Doctor doc ON a.DoctorID = doc.DoctorID;

```
mysql> SELECT p.PatientName, a.AppointmentDate, a.Reason, doc.DoctorName, a.Status
    -> FROM Appointment a
    -> JOIN Patient p ON a.PatientID = p.PatientID
    -> JOIN Doctor doc ON a.DoctorID = doc.DoctorID;
+-------------+-----------------+------------+------------------+-----------+
| PatientName | AppointmentDate | Reason     | DoctorName       | Status    |
+-------------+-----------------+------------+------------------+-----------+
| Rohan Mehta | 2026-10-20      | Chest Pain | Dr. Rajesh Kumar | Confirmed |
| Sneha Rao   | 2026-10-21      | Migraine   | Dr. Priya Sharma | Pending   |
+-------------+-----------------+------------+------------------+-----------+
2 rows in set (0.00 sec)
```

5.1.15. View Creation Query

- **Query/operation description: Create a virtual table (View) combining doctor details with their respective department names.**

SQL code:

- **CREATE VIEW DoctorDeptView AS SELECT d.DoctorName, d.Specialization, dept.DepartmentName FROM Doctor d JOIN Department dept ON d.DepartmentID = dept.DepartmentID;**

```
mysql> CREATE VIEW DoctorDeptView AS
    -> SELECT d.DoctorName, d.Specialization, dept.DepartmentName
    -> FROM Doctor d
    -> JOIN Department dept ON d.DepartmentID = dept.DepartmentID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM DoctorDeptView;
+------------------+----------------+----------------+
| DoctorName       | Specialization | DepartmentName |
+------------------+----------------+----------------+
| Dr. Rajesh Kumar | Cardiologist   | Cardiology     |
| Dr. Priya Sharma | Neurologist    | Neurology      |
| Dr. Amit Singh   | Orthopedic     | Orthopedics    |
+------------------+----------------+----------------+
3 rows in set (0.00 sec)
```

5.1.16. LEFT JOIN (Departments & Doctor Count)

- **Query/operation description:** Show all departments including those that currently do not have any assigned doctors.

SQL code:

- SELECT dept.DepartmentName, d.DoctorName FROM Department dept LEFT JOIN Doctor d ON dept.DepartmentID = d.DepartmentID;

```
mysql> SELECT dept.DepartmentName, d.DoctorName
    -> FROM Department dept
    -> LEFT JOIN Doctor d ON dept.DepartmentID = d.DepartmentID;
+-----------------+-----------------+
| DepartmentName  | DoctorName      |
+-----------------+-----------------+
| Cardiology      | Dr. Rajesh Kumar |
| Neurology       | Dr. Priya Sharma |
| Orthopedics     | Dr. Amit Singh  |
| Pediatrics      | NULL            |
| General Medicine | NULL           |
| Cardiology      | NULL            |
| Neurology       | NULL            |
| Orthopedics     | NULL            |
+-----------------+-----------------+
8 rows in set (0.00 sec)
```

5.1.17. SUBQUERY with IN

- **Query/operation description:** Retrieve names of patients who were admitted into an ICU or Private room.

SQL code:

- SELECT PatientName, Phone FROM Patient WHERE PatientID IN (SELECT ad.PatientID FROM Admission ad JOIN Room r ON ad.RoomID = r.RoomID WHERE r.RoomType IN ('ICU', 'Private'));

```
mysql> SELECT PatientName, Phone FROM Patient
    -> WHERE PatientID IN (
    ->     SELECT ad.PatientID FROM Admission ad
    ->     JOIN Room r ON ad.RoomID = r.RoomID
    ->     WHERE r.RoomType IN ('ICU', 'Private')
    -> );
+-------------+------------+
| PatientName | Phone      |
+-------------+------------+
| Rohan Mehta | 9111222333 |
| Karan Patel | 9345678901 |
+-------------+------------+
2 rows in set (0.04 sec)
```

5.1.18. SUBQUERY with NOT IN

- **Query/operation description:** Find patients who currently do not have any logged medical records.

SQL code:

- SELECT PatientName, Email FROM Patient WHERE PatientID NOT IN (SELECT DISTINCT PatientID FROM MedicalRecord);

```
mysql> SELECT PatientName, Email FROM Patient
    -> WHERE PatientID NOT IN (
    ->     SELECT DISTINCT PatientID FROM MedicalRecord
    -> );
Empty set (0.04 sec)
```

5.1.19. AGGREGATE FUNCTION (SUM & GROUP BY)

- **Query/operation description:** Calculate total billing amount collected per payment method.

SQL code:

- SELECT PaymentMethod, SUM(TotalAmount) AS TotalCollected FROM Bill GROUP BY PaymentMethod;

```
mysql> SELECT PaymentMethod, SUM(TotalAmount) AS TotalCollected
    -> FROM Bill
    -> GROUP BY PaymentMethod;
+---------------+----------------+
| PaymentMethod | TotalCollected |
+---------------+----------------+
| Cash          |        6600.00 |
| Card          |        3600.00 |
| UPI           |        7000.00 |
+---------------+----------------+
3 rows in set (0.00 sec)
```

5.1.20. AGGREGATE FUNCTION (AVG & ROUND)

- **Query/operation description:** Compute the average daily room charges rounded to 2 decimal places.

SQL code:

- SELECT RoomType, ROUND(AVG(ChargesPerDay), 2) AS AverageDailyCharge FROM Room GROUP BY RoomType;

```
mysql> SELECT RoomType, ROUND(AVG(ChargesPerDay), 2) AS AverageDailyCharge
    -> FROM Room
    -> GROUP BY RoomType;
+----------+--------------------+
| RoomType | AverageDailyCharge |
+----------+--------------------+
| General  |            1000.00 |
| ICU      |            5000.00 |
| Private  |            3000.00 |
+----------+--------------------+
3 rows in set (0.01 sec)
```

5.1.21. String Manipulation Query

- **Query/operation description: Format doctor names and specializations into a single descriptive profile string.**

SQL code:

- **SELECT CONCAT(UPPER(DoctorName), ' - Specialization: ', Specialization) AS DoctorProfile FROM Doctor;**

```
mysql> SELECT CONCAT(UPPER(DoctorName), ' - Specialization: ', Specialization) AS DoctorProfile
    -> FROM Doctor;
+-----------------------------------------------+
| DoctorProfile                                 |
+-----------------------------------------------+
| DR. RAJESH KUMAR - Specialization: Cardiologist |
| DR. PRIYA SHARMA - Specialization: Neurologist  |
| DR. AMIT SINGH - Specialization: Orthopedic     |
+-----------------------------------------------+
3 rows in set (0.00 sec)
```

5.1.22. WILDCARD (LIKE Pattern Matching)

- **Query/operation description:** Search for patients whose city/address contains 'abad' or 'lore'.

SQL code:

- SELECT PatientName, Address FROM Patient WHERE Address LIKE '%bad%' OR Address LIKE '%lore%';

```
mysql> SELECT PatientName, Address
    -> FROM Patient
    -> WHERE Address LIKE '%bad%' OR Address LIKE '%lore%';
+-------------+-----------+
| PatientName | Address   |
+-------------+-----------+
| Rohan Mehta | Hyderabad |
| Sneha Rao   | Bangalore |
+-------------+-----------+
2 rows in set (0.00 sec)
```

5.1.23. CONDITIONAL CASE STATEMENT

- **Query/operation description:** Categorize medicines based on their current quantity in stock as Low, Medium, or Sufficient.

SQL code:

- SELECT MedicineName, QuantityInStock, CASE WHEN QuantityInStock <= 100 THEN 'Low Stock' WHEN QuantityInStock BETWEEN 101 AND 300 THEN
  'Medium Stock' ELSE 'Sufficient Stock' END AS StockStatus FROM Medicine;

```
mysql> SELECT MedicineName, QuantityInStock,
    ->        CASE
    ->            WHEN QuantityInStock <= 100 THEN 'Low Stock'
    ->            WHEN QuantityInStock BETWEEN 101 AND 300 THEN 'Medium Stock'
    ->            ELSE 'Sufficient Stock'
    ->        END AS StockStatus
    -> FROM Medicine;
+-------------+-----------------+-----------------+
| MedicineName | QuantityInStock | StockStatus     |
+-------------+-----------------+-----------------+
| Paracetamol  |             500 | Sufficient Stock |
| Aspirin      |             300 | Medium Stock     |
| Amoxicillin  |             200 | Medium Stock     |
| TestMedicine |             100 | Low Stock        |
+-------------+-----------------+-----------------+
4 rows in set (0.00 sec)
```

5.1.24. ADVANCED DATE FUNCTIONS (DATEDIFF)

- **Query/operation description:** Calculate the total duration (in days) each patient spent admitted in the hospital.

SQL code:

- SELECT AdmissionID, PatientID, AdmissionDate, DischargeDate, DATEDIFF(DischargeDate, AdmissionDate) AS TotalDaysAdmitted FROM Admission WHERE DischargeDate IS NOT NULL;

```
mysql> SELECT AdmissionID, PatientID, AdmissionDate, DischargeDate,
    ->        DATEDIFF(DischargeDate, AdmissionDate) AS TotalDaysAdmitted
    -> FROM Admission
    -> WHERE DischargeDate IS NOT NULL;
+-------------+-----------+---------------+---------------+-------------------+
| AdmissionID | PatientID | AdmissionDate | DischargeDate | TotalDaysAdmitted |
+-------------+-----------+---------------+---------------+-------------------+
|           1 |         1 | 2026-10-20    | 2026-10-25    |                 5 |
|           3 |         3 | 2026-10-22    | 2026-10-30    |                 8 |
+-------------+-----------+---------------+---------------+-------------------+
2 rows in set (0.01 sec)
```

5.1.25. COMBINED AGGREGATE & HAVING (High-Value Billing Filter)

- **Query/operation description:** List patients whose total billed amount exceeds 5000.00 across all charges.

SQL code:

- SELECT p.PatientName, SUM(b.TotalAmount) AS TotalBilled FROM Bill b JOIN Patient p ON b.PatientID = p.PatientID GROUP BY p.PatientName HAVING SUM(b.TotalAmount) > 5000.00;

```
mysql> SELECT p.PatientName, SUM(b.TotalAmount) AS TotalBilled
    -> FROM Bill b
    -> JOIN Patient p ON b.PatientID = p.PatientID
    -> GROUP BY p.PatientName
    -> HAVING SUM(b.TotalAmount) > 5000.00;
+--------------+-------------+
| PatientName | TotalBilled |
+--------------+-------------+
| Rohan Mehta |     6600.00 |
| Karan Patel |     7000.00 |
+--------------+-------------+
2 rows in set (0.00 sec)
```